EXPERIMENT 7

Write a Python Program to Implement Breadth-First Search (BFS) Algorithm

Aim

To develop a Python program to implement the **Breadth-First Search (BFS)** algorithm for traversing a graph and visiting all reachable nodes in a level-by-level manner.

Objective

- To understand the concept of **Breadth-First Search (BFS)**.
- To implement BFS using a queue.
- To traverse all nodes of a graph systematically.
- To study graph traversal techniques used in Artificial Intelligence.

Theory

Breadth-First Search (BFS) is an **uninformed search algorithm** used for traversing or searching graphs and trees. It starts from a source node and explores all neighboring nodes before moving to the next level.

BFS uses a **Queue (FIFO - First In First Out)** data structure. It guarantees finding the shortest path in an unweighted graph because nodes are explored level by level.

The basic working of BFS is:

- Start from the source node.
- Mark the source node as visited.
- Insert the source node into the queue.
- Remove the front node from the queue.
- Visit all its unvisited adjacent nodes.
- Add the adjacent nodes to the queue.
- Repeat the process until the queue becomes empty.

Algorithm

Step 1: Create a graph using an adjacency list.

Step 2: Select the starting node.

Step 3: Create an empty queue.

Step 4: Mark the starting node as visited.

Step 5: Insert the starting node into the queue.

Step 6: Remove the front node from the queue.

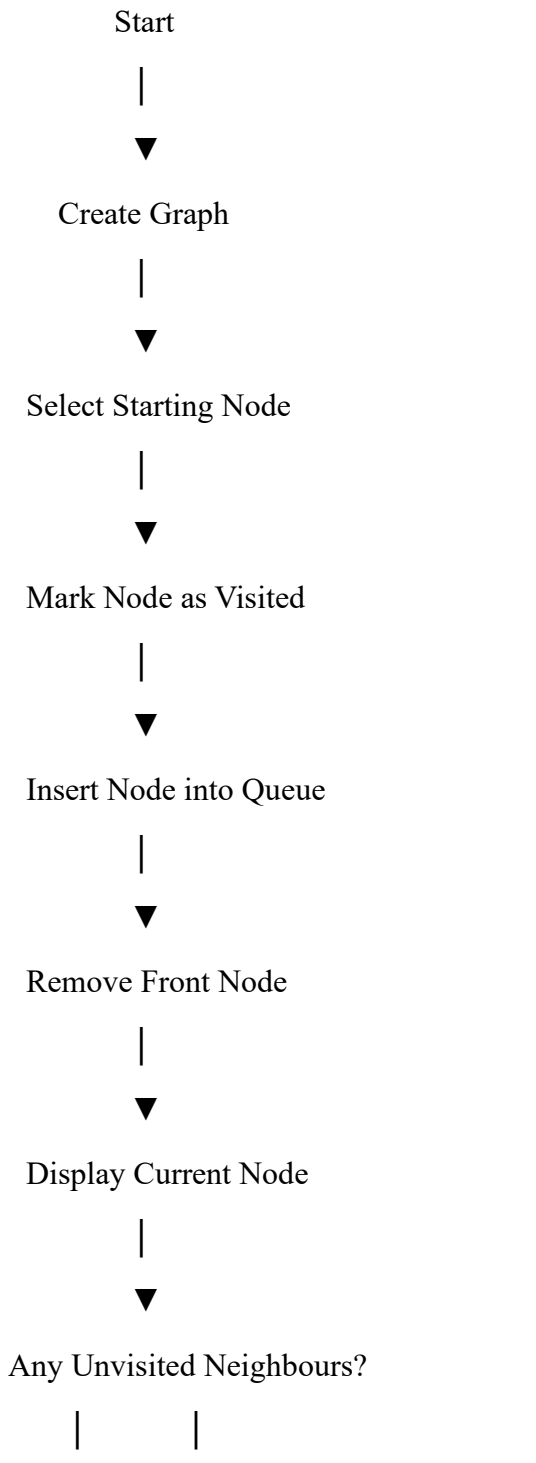
Step 7: Display the current node.

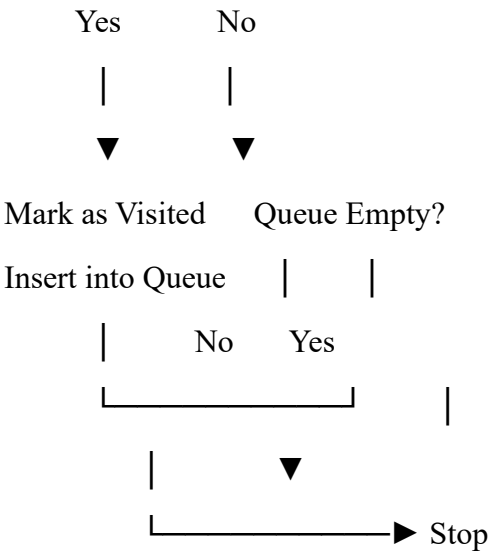
Step 8: Visit all adjacent unvisited nodes.

Step 9: Mark each adjacent node as visited and insert it into the queue.

Step 10: Repeat Steps 6 to 9 until the queue becomes empty.

Flowchart





Python Program

```
from collections import deque


graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}


def bfs(graph, start):

    visited = set()
    queue = deque()

    visited.add(start)
    queue.append(start)
```

```
    while queue:

        node = queue.popleft()
        print(node, end=" ")

        for neighbour in graph[node]:

            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

print("Breadth First Traversal:")

bfs(graph, 'A')
```

Sample Output

Breadth First Traversal:

A B C D E F

Explanation

Consider the graph:

- A is connected to B and C.
- B is connected to D and E.
- C is connected to F.
- E is connected to F.

The BFS traversal starts from node **A**.

Traversal order:

A → B → C → D → E → F

Each node is visited exactly once, and nodes are explored level by level using a queue.

Advantages

- Guarantees the shortest path in an unweighted graph.
- Visits every reachable node.
- Simple and easy to implement.
- Suitable for graph and tree traversal.
- Complete search algorithm.

Applications

- Artificial Intelligence search problems.
- Social networking applications.
- GPS navigation systems.
- Web crawlers used by search engines.
- Network broadcasting.
- Shortest path problems in unweighted graphs.

Result

The Python program successfully implemented the **Breadth-First Search (BFS)** algorithm and traversed all nodes of the graph in **level-order traversal**.

Conclusion

The **Breadth-First Search (BFS)** algorithm is one of the most important graph traversal algorithms in Artificial Intelligence. It systematically explores nodes level by level and guarantees the shortest path in unweighted graphs. The use of a **Queue (FIFO)** ensures efficient traversal without revisiting nodes. This experiment demonstrates the practical implementation of BFS and its applications in AI, networking, robotics, and pathfinding problems.

```
PS C:\Users\Varshini M\AppData\Local\Programs\Microsoft VS C
\python.exe" "c:/Users/Varshini M/OneDrive/Desktop/Here!!!/A
Breadth First Traversal:
A B C D E F
```

```python
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

def bfs(graph, start):

    visited = set()
    queue = deque()

    visited.add(start)
    queue.append(start)

    while queue:

        node = queue.popleft()
        print(node, end=" ")

        for neighbour in graph[node]:

            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

print("Breadth First Traversal:")

bfs(graph, 'A')

```